

OOP

2.0

Sugeneruota Doxygen 1.9.8

1 Hierarchijos Indeksas	1
1.1 Klasių hierarchija	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Klasės Dokumentacija	7
4.1 Studentas Klasė	7
4.2 Vector< T > Klasė Šablonas	7
4.2.1 Tipų Aprašymo Dokumentacija	8
4.2.1.1 const_iterator	8
4.2.1.2 const_pointer	8
4.2.1.3 const_reference	9
4.2.1.4 const_reverse_iterator	9
4.2.1.5 difference_type	9
4.2.1.6 iterator	9
4.2.1.7 pointer	9
4.2.1.8 reference	9
4.2.1.9 reverse_iterator	9
4.2.1.10 size_type	9
4.2.1.11 value_type	9
4.2.2 Konstruktorius ir Destruktorius Dokumentacija	10
4.2.2.1 Vector() [1/4]	10
4.2.2.2 Vector() [2/4]	10
4.2.2.3 Vector() [3/4]	10
4.2.2.4 Vector() [4/4]	10
4.2.2.5 ~Vector()	10
4.2.3 Metodų Dokumentacija	10
4.2.3.1 assign()	10
4.2.3.2 at() [1/2]	10
4.2.3.3 at() [2/2]	11
4.2.3.4 back() [1/2]	11
4.2.3.5 back() [2/2]	11
4.2.3.6 begin() [1/2]	11
4.2.3.7 begin() [2/2]	11
4.2.3.8 capacity()	11
4.2.3.9 cbegin()	11
4.2.3.10 cend()	11
4.2.3.11 clear()	11
4.2.3.12 data() [1/2]	12

4.2.3.13 data() [2/2]	12
4.2.3.14 empty()	12
4.2.3.15 end() [1/2]	12
4.2.3.16 end() [2/2]	12
4.2.3.17 erase() [1/2]	12
4.2.3.18 erase() [2/2]	12
4.2.3.19 front() [1/2]	12
4.2.3.20 front() [2/2]	13
4.2.3.21 insert() [1/2]	13
4.2.3.22 insert() [2/2]	13
4.2.3.23 operator=() [1/2]	13
4.2.3.24 operator=() [2/2]	13
4.2.3.25 operator[]() [1/2]	13
4.2.3.26 operator[]() [2/2]	13
4.2.3.27 pop_back()	14
4.2.3.28 push_back()	14
4.2.3.29 reserve()	14
4.2.3.30 resize()	14
4.2.3.31 shrink_to_fit()	14
4.2.3.32 size()	14
4.2.3.33 swap()	14
4.3 Zmogus Klasė	15
4.3.1 Smulkus aprašymas	15
4.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	16
4.3.2.1 Zmogus() [1/4]	16
4.3.2.2 Zmogus() [2/4]	16
4.3.2.3 ~Zmogus()	16
4.3.2.4 Zmogus() [3/4]	16
4.3.2.5 Zmogus() [4/4]	16
4.3.3 Metodų Dokumentacija	17
4.3.3.1 getPavarde()	17
4.3.3.2 getVardas()	17
4.3.3.3 operator=() [1/2]	17
4.3.3.4 operator=() [2/2]	17
4.3.3.5 setPavarde()	18
4.3.3.6 setVardas()	18
4.3.3.7 vidurkis()	18
4.3.4 Atributų Dokumentacija	18
4.3.4.1 pavarde_	18
4.3.4.2 vardas_	18
5 Failo Dokumentacija	19

5.1 include/funkcijos.h Failo Nuoroda	19
5.1.1 Funkcijos Dokumentacija	20
5.1.1.1 atskiriFailai()	20
5.1.1.2 failoGeneravimas()	20
5.1.1.3 failoSpausdinimas()	21
5.1.1.4 KonteineriuTyrimas()	21
5.1.1.5 rikiavimas()	21
5.1.1.6 skaitymas()	22
5.1.1.7 spausdinimas()	22
5.1.1.8 StudentuPadalinimas()	22
5.1.1.9 StudentuPadalinimas2()	23
5.1.1.10 StudentuPadalinimas3()	23
5.1.1.11 tyrimasAntras()	23
5.1.1.12 tyrimasPirmas()	24
5.2 funkcijos.h	24
5.3 include/studentas.h Failo Nuoroda	28
5.4 studentas.h	28
5.5 include/vector.h Failo Nuoroda	30
5.5.1 Funkcijos Dokumentacija	30
5.5.1.1 operator"!=()	30
5.5.1.2 operator==()	30
5.6 vector.h	31
5.7 include/zmogus.h Failo Nuoroda	35
5.8 zmogus.h	35
5.9 src/funkcijos.cpp Failo Nuoroda	36
5.9.1 Funkcijos Dokumentacija	37
5.9.1.1 failoGeneravimas()	37
5.9.1.2 failoSpausdinimas()	37
5.9.1.3 spausdinimas()	37
5.9.1.4 tyrimasAntras()	37
5.9.1.5 tyrimasPirmas()	38
5.10 src/main.cpp Failo Nuoroda	38
5.10.1 Funkcijos Dokumentacija	38
5.10.1.1 main()	38
5.10.2 Kintamojo Dokumentacija	39
5.10.2.1 pavardes	39
5.10.2.2 vardai	39
Rodyklė	41

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Vector< T >	7
Vector< int >	7
Zmogus	15
Studentas	7

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	
Studento klasė su įvertinimais	7
Vector< T >	7
Zmogus	
Abstrakti klasė sauganti vardą ir pavardę	15

skyrius 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/funkcijos.h	19
include/studentas.h	28
include/vector.h	30
include/zmogus.h	35
src/funkcijos.cpp	36
src/main.cpp	38

skyrius 4

Klasės Dokumentacija

4.1 Studentas Klasė

Studento klasė su įvertinimais.

```
#include <studentas.h>
```

Paveldimumo diagrama Studentas:

4.2 Vector< T > Klasė Šablonas

```
#include <vector.h>
```

Vieši Tipai

- `using value_type = T`
- `using reference = value_type &`
- `using const_reference = const value_type &`
- `using size_type = std::size_t`
- `using difference_type = std::ptrdiff_t`
- `using pointer = T *`
- `using const_pointer = const T *`
- `using iterator = T *`
- `using const_iterator = const T *`
- `using reverse_iterator = std::reverse_iterator< iterator >`
- `using const_reverse_iterator = std::reverse_iterator< const_iterator >`

Vieši Metodai

- `Vector()` = default
- `Vector(const Vector &other)`
- `Vector(Vector &&other) noexcept`
- `Vector(std::initializer_list< value_type > init)`
- `~Vector()`
- `Vector & operator= (const Vector &other)`
- `Vector & operator= (Vector &&other) noexcept`
- `const_reference operator[] (size_type index) const`
- `reference operator[] (size_type index)`
- `void resize (size_type newSize)`
- `void assign (size_type count, const_reference value)`
- `void push_back (const_reference value)`
- `void pop_back ()`
- `void shrink_to_fit ()`
- `void clear ()`
- `iterator insert (const_iterator pos, const_reference value)`
- `template<typename InputIt >`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
- `iterator erase (const_iterator pos)`
- `iterator erase (const_iterator first, const_iterator last)`
- `void swap (Vector &other)`
- `pointer data ()`
- `const_pointer data () const`
- `reference at (size_type index)`
- `const_reference at (size_type index) const`
- `reference front ()`
- `const_reference front () const`
- `reference back ()`
- `const_reference back () const`
- `bool empty () const`
- `size_type size () const`
- `size_type capacity () const`
- `void reserve (size_type new_cap)`
- `iterator begin ()`
- `const_iterator begin () const`
- `iterator end ()`
- `const_iterator end () const`
- `const_iterator cbegin () const noexcept`
- `const_iterator cend () const noexcept`

4.2.1 Tipo Aprašymo Dokumentacija

4.2.1.1 const_iterator

```
template<typename T >
using Vector< T >::const_iterator = const T*
```

4.2.1.2 const_pointer

```
template<typename T >
using Vector< T >::const_pointer = const T*
```

4.2.1.3 const_reference

```
template<typename T >
using Vector< T >::const_reference = const value_type&
```

4.2.1.4 const_reverse_iterator

```
template<typename T >
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

4.2.1.5 difference_type

```
template<typename T >
using Vector< T >::difference_type = std::ptrdiff_t
```

4.2.1.6 iterator

```
template<typename T >
using Vector< T >::iterator = T*
```

4.2.1.7 pointer

```
template<typename T >
using Vector< T >::pointer = T*
```

4.2.1.8 reference

```
template<typename T >
using Vector< T >::reference = value_type&
```

4.2.1.9 reverse_iterator

```
template<typename T >
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

4.2.1.10 size_type

```
template<typename T >
using Vector< T >::size_type = std::size_t
```

4.2.1.11 value_type

```
template<typename T >
using Vector< T >::value_type = T
```

4.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.2.2.1 Vector() [1/4]

```
template<typename T >
Vector< T >::Vector ( ) [default]
```

4.2.2.2 Vector() [2/4]

```
template<typename T >
Vector< T >::Vector (
    const Vector< T > & other ) [inline]
```

4.2.2.3 Vector() [3/4]

```
template<typename T >
Vector< T >::Vector (
    Vector< T > && other ) [inline], [noexcept]
```

4.2.2.4 Vector() [4/4]

```
template<typename T >
Vector< T >::Vector (
    std::initializer_list< value_type > init ) [inline]
```

4.2.2.5 ~Vector()

```
template<typename T >
Vector< T >::~~Vector ( ) [inline]
```

4.2.3 Metodų Dokumentacija

4.2.3.1 assign()

```
template<typename T >
void Vector< T >::assign (
    size_type count,
    const_reference value ) [inline]
```

4.2.3.2 at() [1/2]

```
template<typename T >
reference Vector< T >::at (
    size_type index ) [inline]
```


4.2.3.3 at() [2/2]

```
template<typename T >
const_reference Vector< T >::at (
    size_type index ) const [inline]
```

4.2.3.4 back() [1/2]

```
template<typename T >
reference Vector< T >::back ( ) [inline]
```

4.2.3.5 back() [2/2]

```
template<typename T >
const_reference Vector< T >::back ( ) const [inline]
```

4.2.3.6 begin() [1/2]

```
template<typename T >
iterator Vector< T >::begin ( ) [inline]
```

4.2.3.7 begin() [2/2]

```
template<typename T >
const_iterator Vector< T >::begin ( ) const [inline]
```

4.2.3.8 capacity()

```
template<typename T >
size_type Vector< T >::capacity ( ) const [inline]
```

4.2.3.9 cbegin()

```
template<typename T >
const_iterator Vector< T >::cbegin ( ) const [inline], [noexcept]
```

4.2.3.10 cend()

```
template<typename T >
const_iterator Vector< T >::cend ( ) const [inline], [noexcept]
```

4.2.3.11 clear()

```
template<typename T >
void Vector< T >::clear ( ) [inline]
```

4.2.3.12 data() [1/2]

```
template<typename T >
pointer Vector< T >::data ( ) [inline]
```

4.2.3.13 data() [2/2]

```
template<typename T >
const_pointer Vector< T >::data ( ) const [inline]
```

4.2.3.14 empty()

```
template<typename T >
bool Vector< T >::empty ( ) const [inline]
```

4.2.3.15 end() [1/2]

```
template<typename T >
iterator Vector< T >::end ( ) [inline]
```

4.2.3.16 end() [2/2]

```
template<typename T >
const_iterator Vector< T >::end ( ) const [inline]
```

4.2.3.17 erase() [1/2]

```
template<typename T >
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last ) [inline]
```

4.2.3.18 erase() [2/2]

```
template<typename T >
iterator Vector< T >::erase (
    const_iterator pos ) [inline]
```

4.2.3.19 front() [1/2]

```
template<typename T >
reference Vector< T >::front ( ) [inline]
```

4.2.3.20 front() [2/2]

```
template<typename T >
const_reference Vector< T >::front ( ) const [inline]
```

4.2.3.21 insert() [1/2]

```
template<typename T >
iterator Vector< T >::insert (
    const_iterator pos,
    const_reference value ) [inline]
```

4.2.3.22 insert() [2/2]

```
template<typename T >
template<typename InputIt >
iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last ) [inline]
```

4.2.3.23 operator=() [1/2]

```
template<typename T >
Vector & Vector< T >::operator= (
    const Vector< T > & other ) [inline]
```

4.2.3.24 operator=() [2/2]

```
template<typename T >
Vector & Vector< T >::operator= (
    Vector< T > && other ) [inline], [noexcept]
```

4.2.3.25 operator[]() [1/2]

```
template<typename T >
reference Vector< T >::operator[] (
    size_type index ) [inline]
```

4.2.3.26 operator[]() [2/2]

```
template<typename T >
const_reference Vector< T >::operator[] (
    size_type index ) const [inline]
```

4.2.3.27 pop_back()

```
template<typename T >
void Vector< T >::pop_back ( ) [inline]
```

4.2.3.28 push_back()

```
template<typename T >
void Vector< T >::push_back (
    const_reference value ) [inline]
```

4.2.3.29 reserve()

```
template<typename T >
void Vector< T >::reserve (
    size_type new_cap ) [inline]
```

4.2.3.30 resize()

```
template<typename T >
void Vector< T >::resize (
    size_type newSize ) [inline]
```

4.2.3.31 shrink_to_fit()

```
template<typename T >
void Vector< T >::shrink_to_fit ( ) [inline]
```

4.2.3.32 size()

```
template<typename T >
size_type Vector< T >::size ( ) const [inline]
```

4.2.3.33 swap()

```
template<typename T >
void Vector< T >::swap (
    Vector< T > & other ) [inline]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- include/[vector.h](#)

4.3 Zmogus Klasė

Abstrakti klasė sauganti vardą ir pavardę

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:

Vieši Metodai

- `Zmogus()`=default
Default konstruktorius.
- `Zmogus (const std::string &v, const std::string &p)`
konstruktorius su parametrais
- `virtual ~Zmogus ()`
Virtualus destruktorius.
- `Zmogus (const Zmogus &a)`
Kopijos konstruktorius.
- `Zmogus & operator= (const Zmogus &a)`
Kopijos priskyrimo operatorius.
- `Zmogus (Zmogus &&a)`
Perkėlimo konstruktorius.
- `Zmogus & operator= (Zmogus &&a)`
Perkėlimo priskyrimo operatorius.
- `virtual double vidurkis () const =0`
Grynai virtualus vidurkio metodas Paveldinčios klasės privalo realizuoti šį metodą
- `const std::string & getVardas () const`
Grąžina vardą
- `const std::string & getPavarde () const`
Grąžina pavardę
- `void setVardas (const std::string &v)`
Nustato vardą
- `void setPavarde (const std::string &p)`
Nustato pavardę

Apsaugoti Atributai

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė

4.3.1 Smulkus aprašymas

Abstrakti klasė sauganti vardą ir pavardę

4.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.3.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ( ) [default]
```

Default konstruktorius.

4.3.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p ) [inline]
```

konstruktorius su parametrais

Parametrai

<i>v</i>	Vardas
<i>p</i>	Pavardė

4.3.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus ( ) [inline], [virtual]
```

Virtualus destruktoriaus.

4.3.2.4 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & a ) [inline]
```

Kopijos konstruktorius.

Parametrai

<i>a</i>	Kopijuojamas objektas
----------	-----------------------

4.3.2.5 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && a ) [inline]
```

Perkėlimo konstruktorius.

Parametrai

<i>a</i>	Priskiriamas objektas
----------	-----------------------

4.3.3 Metodų Dokumentacija

4.3.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde ( ) const [inline]
```

Gražina pavardę

4.3.3.2 getVardas()

```
const std::string & Zmogus::getVardas ( ) const [inline]
```

Gražina vardą

4.3.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & a ) [inline]
```

Kopijos priskyrimo operatorius.

Parametrai

<i>a</i>	Priskiriamas objektas
----------	-----------------------

Gražina

Nuoroda į objektą

4.3.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && a ) [inline]
```

Perkėlimo priskyrimo operatorius.

Parametrai

<i>a</i>	Perkeliamas objektas
----------	----------------------

Gražina

nuoroda į objektą

4.3.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

Nustato pavardę

Parametrai

<i>p</i>	Pavardė
----------	---------

4.3.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

Nustato vardą

Parametrai

<i>v</i>	Vardas
----------	--------

4.3.3.7 vidurkis()

```
virtual double Zmogus::vidurkis ( ) const [pure virtual]
```

Grynai virtualus vidurkio metodus Paveldinčios klasės privalo realizuoti šį metodą

Realizuota [Studentas](#).

4.3.4 Atributų Dokumentacija**4.3.4.1 pavarde_**

```
std::string Zmogus::pavarde_ [protected]
```

Žmogaus pavardė

4.3.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Žmogaus vardas.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [include/zmogus.h](#)

skyrius 5

Failo Dokumentacija

5.1 include/funkcijos.h Failo Nuoroda

```
#include "studentas.h"
#include <iostream>
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <algorithm>
#include <list>
#include <deque>
#include <chrono>
#include <iomanip>
#include "vector.h"
```

Įtraukimo priklausomybių diagrama funkcijos.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Funkcijos

- void [spausdinimas](#) ([Vector](#)< [Studentas](#) > &A)
Spausdina studentų sąrašą
- void [failoSpausdinimas](#) ([Vector](#)< [Studentas](#) > &A)
Spausina studentų sąrašą ekrane arba faile.
- void [failoGeneravimas](#) (int studKiekis)
Generuoja studentų failą
- void [tyrimasPirmas](#) ()
Atlieka tyrimą - matuoja failų generavimo laiką
- void [tyrimasAntras](#) ([Vector](#)< [Studentas](#) > &A, [Vector](#)< [Studentas](#) > &vargsai, [Vector](#)< [Studentas](#) > &kietekai)
Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką
- template<typename konteineris >
void [skaitymas](#) (konteineris &A, std::string failas)
Nuskaito duomenis iš failo.
- template<typename konteineris >
void [atskiriFailai](#) (std::string failas, konteineris &vargsai, konteineris &kietekai)
Surašo vargšus ir kietekus į atskirus failus.

- `template<typename konteineris >`
`void rikiavimas` (konteineris &A, int kriterijus)
Rikiuoja studentus pagal pasirinktą kriterijų
- `template<typename konteineris >`
`void StudentuPadalinimas` (konteineris &A, konteineris &vargsai, konteineris &kietekai)
1 strategija - dalina studentus į du atskirus konteinerius
- `template<typename konteineris >`
`void StudentuPadalinimas2` (konteineris &A, konteineris &vargsai)
2 strategija - rikiuoja ir išima vargšus iš galo
- `template<typename konteineris >`
`void StudentuPadalinimas3` (konteineris &A, konteineris &vargsai)
3 strategija - naudoja partition padalinimui
- `template<typename konteineris >`
`void KonteineriuTyrimas` (konteineris &A, konteineris &vargsai, konteineris &kietekai, int kriterijus, int strategija)
Atlieka skirtingų konteinerių greičio tyrimą su skirtingomis strategijomis.

5.1.1 Funkcijos Dokumentacija

5.1.1.1 atskiriFailai()

```
template<typename konteineris >
void atskiriFailai (
    std::string failas,
    konteineris & vargsai,
    konteineris & kietekai )
```

Surašo vargšus ir kietekus į atskirus failus.

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>failas</i>	Originalaus failo pavadinimas
<i>vargsai</i>	Vargšų studentų konteineris
<i>kietekai</i>	Kietekų studentų konteineris

5.1.1.2 failoGeneravimas()

```
void failoGeneravimas (
    int studKiekis )
```

Generuoja studentų failą

Parametrai

<i>studKiekis</i>	Studentų kiekis
-------------------	-----------------

5.1.1.3 failoSpausdinimas()

```
void failoSpausdinimas (
    Vector< Studentas > & A )
```

Spausina studentų sąrašą ekrane arba faile.

Parametrai

<i>A</i>	Studentų vektorius
----------	--------------------

5.1.1.4 KonteineriuTyrimas()

```
template<typename konteineris >
void KonteineriuTyrimas (
    konteineris & A,
    konteineris & vargsai,
    konteineris & kietekai,
    int kriterijus,
    int strategija )
```

Atlieka skirtingų konteinerių greičio tyrimą su skirtingomis strategijomis.

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris
<i>kietekai</i>	Gerau besimokančių studentų konteineris
<i>kriterijus</i>	Rikiavimo kriterijus (1 - vardas; 2 - pavardė; 3 - galutinis pagal vidurkį; 4 - galutinis pagal medianą)
<i>strategija</i>	Padalinimo strategija (1, 2 arba 3)

```
atskiriFailai("studentai" + std::to_string(x) + ".txt", vargsai, kietekai);
```

5.1.1.5 rikiavimas()

```
template<typename konteineris >
void rikiavimas (
    konteineris & A,
    int kriterijus )
```

Rikiuoja studentus pagal pasirinktą kriterijų

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>kriterijus</i>	Rikiavimo kriterijus (1 - vardas; 2 - pavardė; 3 - galutinis pagal vidurkį; 4 - galutinis pagal medianą)

5.1.1.6 skaitymas()

```
template<typename konteineris >
void skaitymas (
    konteineris & A,
    std::string failas )
```

Nuskaityto duomenis iš failo.

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>failas</i>	Failo pavadinimas

5.1.1.7 spausdinimas()

```
void spausdinimas (
    Vector< Studentas > & A )
```

Spausdina studentų sąrašą

Parametrai

<i>A</i>	Studentų vektorius
----------	--------------------

5.1.1.8 StudentuPadalinimas()

```
template<typename konteineris >
void StudentuPadalinimas (
    konteineris & A,
    konteineris & vargsai,
    konteineris & kietekai )
```

1 strategija - dalina studentus į du atskirus konteinerius

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris
<i>kietekai</i>	Gerau besimokančių studentų konteineris

5.1.1.9 StudentuPadalinimas2()

```
template<typename konteineris >
void StudentuPadalinimas2 (
    konteineris & A,
    konteineris & vargsai )
```

2 strategija - rikiuoja ir išima vargšus iš galo

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris

5.1.1.10 StudentuPadalinimas3()

```
template<typename konteineris >
void StudentuPadalinimas3 (
    konteineris & A,
    konteineris & vargsai )
```

3 strategija - naudoja partition padalinimui

Template Parameters

<i>konteineris</i>	Konteineris (vector, list, deque)
--------------------	-----------------------------------

Parametrai

<i>A</i>	Studentų konteineris
<i>vargsai</i>	Prastai besimokančių studentų konteineris

5.1.1.11 tyrimasAntras()

```
void tyrimasAntras (
    Vector< Studentas > & A,
```

```
Vector< Studentas > & vargsai,
Vector< Studentas > & kietekai )
```

Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

Parametrai

<i>A</i>	Studentų vektorius
<i>vargsai</i>	Prastai besimokančių studentų vektorius
<i>kietekai</i>	Gera besimokančių studentų vektorius

5.1.1.12 tyrimasPirmas()

```
void tyrimasPirmas ( )
```

Atlieka tyrimą - matuoja failų generavimo laiką

5.2 funkcijos.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "studentas.h"
00005 #include <iostream>
00006 #include <string>
00007 #include <vector>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <stdexcept>
00011 #include <algorithm>
00012 #include <list>
00013 #include <deque>
00014 #include <chrono>
00015 #include <iomanip>
00016 #include "vector.h"
00017
00022 void spausdinimas(Vector<Studentas>& A);
00023
00028 void failoSpausdinimas(Vector<Studentas>& A);
00029
00034 void failoGeneravimas(int studKiekis);
00035
00039 void tyrimasPirmas();
00040
00047 void tyrimasAntras(Vector<Studentas>& A, Vector<Studentas>& vargsai, Vector<Studentas>& kietekai);
00048
00049
00056 template<typename konteineris>
00057 void skaitymas(konteineris& A, std::string failas)
00058 {
00059     std::ifstream input(failas);
00060
00061     if(!input.is_open())
00062     {
00063         throw std::runtime_error("Pasirinkto failo nepavyko atidaryti!");
00064     }
00065     std::string eilute;
00066
00067     getline(input, eilute);
00068
00069     while(getline(input, eilute))
00070     {
00071         std::stringstream ss(eilute);
00072         Studentas s;
00073         std::string vardas, pavarde;
00074         ss » vardas » pavarde;
00075         s.setVardas(vardas);
```

```

00076         s.setPavarde(pavarde);
00077
00078         Vector<int> pazymiai;
00079         int x;
00080         while(ss >> x)
00081         {
00082             pazymiai.push_back(x);
00083         }
00084
00085         s.setEgzaminas(pazymiai.back());
00086         pazymiai.pop_back();
00087
00088
00089         s.setNd(pazymiai);
00090         s.setGalutinisVid(s.galutinisVid());
00091         s.setGalutinisMed(s.galutinisMed());
00092
00093         A.push_back(s);
00094     }
00095     input.close();
00096 }
00097
00105 template<typename konteineris>
00106 void atskiriFailai(std::string failas, konteineris& vargsai, konteineris& kietekai)
00107 {
00108     std::ofstream vargsuf("vargsai_" + failas);
00109     std::ofstream kietekuf("kietekai_" + failas);
00110
00111     vargsuf << std::left << std::setw(10) << "Vardas" << std::setw(10) << "Pavarde"
00112     << std::setw(20) << "Galutinis(Vid.)" << std::setw(20) << "Galutinis(Med.)" << "\n";
00113
00114     kietekuf << std::left << std::setw(15) << "Vardas" << std::setw(20) << "Pavarde"
00115     << std::setw(20) << "Galutinis(Vid.)" << std::setw(20) << "Galutinis(Med.)" << "\n";
00116
00117     for(auto& s : vargsai)
00118     {
00119         vargsuf << std::setw(15) << s.getVardas() << std::setw(20) << s.getPavarde();
00120
00121         vargsuf << std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisVid() <<
00122         std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisMed() << "\n";
00123     }
00124
00125     if(!kietekai.empty())
00126     {
00127         for(auto& s : kietekai)
00128         {
00129             kietekuf << std::setw(15) << s.getVardas() << std::setw(20) << s.getPavarde();
00130
00131             kietekuf << std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisVid() <<
00132             std::setw(20) << std::fixed << std::setprecision(2) << s.getGalutinisMed() << "\n";
00133         }
00134     }
00135 }
00136
00143 template<typename konteineris>
00144 void rikiavimas(konteineris& A, int kriterijus)
00145 {
00146
00147     auto byVardas = [](const Studentas& a, const Studentas& b)
00148     {
00149         return a.getVardas() < b.getVardas();
00150     };
00151     auto byPavarde = [](const Studentas& a, const Studentas& b)
00152     {
00153         return a.getPavarde() < b.getPavarde();
00154     };
00155     auto byGalVid = [](const Studentas& a, const Studentas& b)
00156     {
00157         return a.getGalutinisVid() < b.getGalutinisVid();
00158     };
00159     auto byGalMed = [](const Studentas& a, const Studentas& b)
00160     {
00161         return a.getGalutinisMed() < b.getGalutinisMed();
00162     };
00163
00164     //Listo rikiavimas
00165     if constexpr (std::is_same_v<konteineris, std::list<Studentas> >)
00166     {
00167         switch(kriterijus)
00168         {
00169             case 1: A.sort(byVardas); break;
00170             case 2: A.sort(byPavarde); break;
00171             case 3: A.sort(byGalVid); break;
00172             case 4: A.sort(byGalMed); break;
00173         }
00174     }

```

```

00174     }
00175     //Vector arba deque rikiavimas
00176     else
00177     {
00178         switch(kriterijus)
00179         {
00180             case 1: std::sort(A.begin(), A.end(), byVardas); break;
00181             case 2: std::sort(A.begin(), A.end(), byPavarde); break;
00182             case 3: std::sort(A.begin(), A.end(), byGalVid); break;
00183             case 4: std::sort(A.begin(), A.end(), byGalMed); break;
00184         }
00185     }
00186 }
00187 }
00188 }
00189
00190 template<typename konteineris>
00191 void StudentuPadalinimas( konteineris& A, konteineris& vargsai, konteineris& kietekai)
00192 {
00193     for(const auto& s : A)
00194     {
00195         if(s.getGalutinisVid() < 5.0)
00196         {
00197             vargsai.push_back(s);
00198         }
00199         else
00200         {
00201             kietekai.push_back(s);
00202         }
00203     }
00204     A.clear();
00205 }
00206
00207 template<typename konteineris>
00208 void StudentuPadalinimas2(konteineris& A, konteineris& vargsai)
00209 {
00210     if constexpr (std::is_same_v<konteineris, std::list<Studentas>>)
00211     {
00212         A.sort([](const Studentas& a, const Studentas& b){
00213             return a.getGalutinisVid() > b.getGalutinisVid();
00214         });
00215     }
00216     else
00217     {
00218         std::sort(A.begin(), A.end(), [](const Studentas& a, const Studentas& b){
00219             return a.getGalutinisVid() > b.getGalutinisVid();
00220         });
00221     }
00222     while(A.back().galutinisVid() < 5)
00223     {
00224         vargsai.push_back(A.back());
00225         A.pop_back();
00226     }
00227     /*
00228     auto it = A.end();
00229     while(it != A.begin())
00230     {
00231         --it;
00232         if(it->getGalutinisVid() < 5)
00233         {
00234             vargsai.push_back(*it);
00235             it = A.erase(it);
00236         }
00237         else{
00238             break;
00239         }
00240     }
00241     */
00242
00243     /*remove_if iskelia konteinerio elementus i jo gala, ties kuriais lambda grazina true
00244     // it yra iteratorius kuri po remove_if rodo i pirma "netikusi" elementa, kuri reikia pasalinti
00245     auto it = remove_if(A.begin(), A.end(), [&vargsai](studentas& a)
00246     {
00247         if(a.galutinisVid < 5.0)
00248         {
00249             vargsai.push_back(a);
00250             return true;
00251         }
00252         return false;
00253     });
00254     //istrinam pagrindinio konteinerio galo elementus, kurie po remove_if buvo ten sudėti
00255     A.erase(it, A.end());
00256     */
00257 }
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270
00271
00272
00273

```



```

00274 }
00275
00282 template<typename konteineris>
00283 void StudentuPadalinimas3(konteineris& A, konteineris& vargsai)
00284 {
00285     //grazina iteratoriu it, kuris rodo i antro bloko pirma elementa, kuris pagal partition ir musu
    salyga buvo false
00286     //konteineris lieka isdeliotas taip: pirmi elementai tie, kurie tenkina salyga, tada po ju eina
    visi like
00287     auto it = std::partition(A.begin(), A.end(), [](const Studentas& a) { return a.getGalutinisVid()
    >= 5.0; });
00288
00289     //sudeda elementus nuo konteinerio A iteratoriaus iki konteinerio pabaigos i vargsai konteineri
00290     vargsai.insert(vargsai.begin(), it, A.end());
00291     //istrina A konteinerio elementus nuo iteratoriaus iki konteinerio pabaigos
00292     A.erase(it, A.end());
00293
00294 }
00295
00305 template<typename konteineris>
00306 void KonteineriuTyrimas(konteineris& A, konteineris& vargsai, konteineris& kietekai, int kriterijus,
    int strategija)
00307 {
00308     Vector<int> studKiekis = {1000, 10000, 100000, 1000000, 10000000};
00309
00310     std::cout << std::left << std::setw(12) << "Studentai" << std::setw(15) << "Skaitymas"
00311     << std::setw(15) << "Rikiavimas" << std::setw(15) << "Skirstymas"
00312     << std::setw(15) << "Bendras laikas" << std::endl;
00313
00314
00315     for(auto x : studKiekis)
00316     {
00317         A.clear();
00318         vargsai.clear();
00319         kietekai.clear();
00320
00321         //nuskaitymas
00322         auto start1 = std::chrono::high_resolution_clock::now();
00323         skaitymas(A, "studentai" + std::to_string(x) + ".txt");
00324         auto end1 = std::chrono::high_resolution_clock::now();
00325
00326         std::chrono::duration<double> diff1 = end1 - start1;
00327
00328         //rikiavimas
00329         auto start2 = std::chrono::high_resolution_clock::now();
00330         rikiavimas(A, kriterijus);
00331         auto end2 = std::chrono::high_resolution_clock::now();
00332
00333         std::chrono::duration<double> diff2 = end2 - start2;
00334
00335         //padalinimas
00336         auto start3 = std::chrono::high_resolution_clock::now();
00337         if(strategija == 1)
00338         {
00339             StudentuPadalinimas(A, vargsai, kietekai);
00340         }
00341         else if(strategija == 2)
00342         {
00343             StudentuPadalinimas2(A, vargsai);
00344             //kietekai = A;
00345             //A.clear();
00346         }
00347         else if(strategija == 3)
00348         {
00349             StudentuPadalinimas3(A, vargsai);
00350             //kietekai = A;
00351             //A.clear();
00352         }
00353         auto end3 = std::chrono::high_resolution_clock::now();
00354
00355         std::chrono::duration<double> diff3 = end3 - start3;
00356
00357         //bendras
00358         double visas = diff1.count() + diff2.count() + diff3.count();
00359
00360         std::cout << std::left << std::setw(12) << x << std::setw(15) << diff1.count()
00361         << std::setw(15) << diff2.count() << std::setw(15) << diff3.count()
00362         << std::setw(15) << visas << std::endl;
00363
00364
00365
00366
00367
00368
00369     }
00370     A.clear();
00371     vargsai.clear();
00372     kietekai.clear();

```

```
00373 }
00374
00375 #endif
```

5.3 include/studentas.h Failo Nuoroda

```
#include <string>
#include <vector>
#include <sstream>
#include <iostream>
#include "zmogus.h"
#include "vector.h"
```

Įtraukimo priklausomybių diagrama studentas.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Klasės

- class [Studentas](#)
Studento klasė su įvertinimais.

5.4 studentas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <sstream>
00007 #include <iostream>
00008 #include "zmogus.h"
00009 #include "vector.h"
00010
00019 class Studentas : public Zmogus{
00020
00021 private:
00022     Vector<int> nd_;
00023     int egzaminas_;
00024     double galutinisVid_;
00025     double galutinisMed_;
00026
00027 public:
00031     Studentas()
00032         : Zmogus(), egzaminas_(0), galutinisVid_(0.0), galutinisMed_(0.0) {}
00033
00039     Studentas(const std::string& v, const std::string& p)
00040         : Zmogus(v, p), egzaminas_(0), galutinisVid_(0.0), galutinisMed_(0.0) {}
00041
00045     ~Studentas() override
00046     {
00047         vardas_.clear();
00048         pavarde_.clear();
00049         nd_.clear();
00050         egzaminas_ = 0;
00051         galutinisVid_ = 0.0;
00052         galutinisMed_ = 0.0;
00053     }
00054
00055
00060     Studentas(const Studentas &a)
00061         : Zmogus(a),
00062           nd_(a.nd_),
00063           egzaminas_(a.egzaminas_),
00064           galutinisVid_(a.galutinisVid_),
00065           galutinisMed_(a.galutinisMed_) {}
00066
00067
```

```

00073     Studentas& operator=(const Studentas &a)
00074     {
00075         if(this != &a)
00076         {
00077             Zmogus::operator=(a);
00078             nd_ = a.nd_;
00079             egzaminas_ = a.egzaminas_;
00080             galutinisVid_ = a.galutinisVid_;
00081             galutinisMed_ = a.galutinisMed_;
00082         }
00083         return *this;
00084     }
00085 }
00086
00091 Studentas (Studentas&& a)
00092 : Zmogus(std::move(a)),
00093   nd_(std::move(a.nd_)),
00094   egzaminas_(std::move(a.egzaminas_)),
00095   galutinisVid_(std::move(a.galutinisVid_)),
00096   galutinisMed_(std::move(a.galutinisMed_))
00097 {
00098     a.egzaminas_ = 0;
00099     a.galutinisVid_ = 0.0;
00100     a.galutinisMed_ = 0.0;
00101 }
00102
00108 Studentas& operator=(Studentas&& a)
00109 {
00110     if(this != &a)
00111     {
00112         Zmogus::operator=(std::move(a));
00113         nd_ = std::move(a.nd_);
00114         egzaminas_ = std::move(a.egzaminas_);
00115         galutinisVid_ = std::move(a.galutinisVid_);
00116         galutinisMed_ = std::move(a.galutinisMed_);
00117
00118         a.egzaminas_ = 0;
00119         a.galutinisVid_ = 0.0;
00120         a.galutinisMed_ = 0.0;
00121     }
00122     return *this;
00123 }
00124
00126 friend std::ostream& operator<<(std::ostream& out, const Studentas& a)
00127 {
00128     out << a.vardas_ << " " << a.pavarde_ << " ";
00129     for(int i : a.nd_)
00130     {
00131         out << i << " ";
00132     }
00133     out << a.egzaminas_;
00134     return out;
00135 }
00136
00143 friend std::istream& operator>>(std::istream& in, Studentas& a)
00144 {
00145     std::string eilute;
00146     std::getline(in, eilute);
00147     std::stringstream ss(eilute);
00148     ss >> a.vardas_ >> a.pavarde_;
00149
00150     int skaicius;
00151     Vector<int> visi;
00152
00153     while(ss >> skaicius)
00154     {
00155         visi.push_back(skaicius);
00156     }
00157     a.egzaminas_ = visi.back();
00158     visi.pop_back();
00159     a.nd_ = visi;
00160
00161     return in;
00162 }
00163
00166 const Vector<int>& getNd() const { return nd_; }
00167
00169 int getEgzaminas() const { return egzaminas_; }
00170
00172 double getGalutinisVid() const { return galutinisVid_; }
00173
00175 double getGalutinisMed() const { return galutinisMed_; }
00176
00178 void setNd(const Vector<int>& n) { nd_ = n; }
00179
00180
00181
00182
00183
00184
00185
00186
00187
00188
00189
00190
00191
00192
00193
00194
00195
00196

```

```

00201     void setEgzaminas(int e) { egzaminas_ = e; }
00202
00207     void setGalutinisVid(double galVid) { galutinisVid_ = galVid; }
00208
00213     void setGalutinisMed(double galMed) { galutinisMed_ = galMed; }
00214
00219     double vidurkis() const override;
00220
00225     double mediana() const;
00226
00231     double galutinisVid() const;
00232
00237     double galutinisMed() const;
00238
00239
00240 };
00241
00242 #endif

```

5.5 include/vector.h Failo Nuoroda

```

#include <iostream>
#include <cstdint>
#include <iterator>

```

Įtraukimo priklausomybių diagrama vector.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Klasės

- class [Vector< T >](#)

Funkcijos

- template<typename T >
bool [operator==](#) (const [Vector< T >](#) &lhs, const [Vector< T >](#) &rhs)
- template<typename T >
bool [operator!=](#) (const [Vector< T >](#) &lhs, const [Vector< T >](#) &rhs)

5.5.1 Funkcijos Dokumentacija

5.5.1.1 operator"!=()

```

template<typename T >
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs )

```

5.5.1.2 operator==()

```

template<typename T >
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs )

```

5.6 vector.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <iostream>
00005 #include <cstdint>
00006 #include <iterator>
00007
00008
00009 template<typename T>
00010 class Vector
00011 {
00012 private:
00013     T* data_ = nullptr;
00014     size_t size_ = 0;
00015     size_t capacity_ = 0;
00016
00017     void ReAlloc(size_t newCapacity)
00018     {
00019
00020         T* newBlock = new T[newCapacity];
00021
00022         if(newCapacity < size_)
00023         {
00024             size_ = newCapacity;
00025         }
00026
00027         for(size_t i = 0; i < size_; i++)
00028         {
00029             newBlock[i] = std::move(data_[i]);
00030         }
00031         delete[] data_;
00032         data_ = newBlock;
00033         capacity_ = newCapacity;
00034     }
00035
00036 public:
00037     using value_type = T;
00038     using reference = value_type&;
00039     using const_reference = const value_type&;
00040     using size_type = std::size_t;
00041     using difference_type = std::ptrdiff_t;
00042     using pointer = T*;
00043     using const_pointer = const T*;
00044
00045     using iterator = T*;
00046     using const_iterator = const T*;
00047     using reverse_iterator = std::reverse_iterator<iterator>;
00048     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00049
00050     //numatytasis konstruktorius
00051     Vector() = default;
00052     //copy konstruktorius
00053     Vector(const Vector& other)
00054         :size_(other.size_),
00055         capacity_(other.capacity_)
00056     {
00057         data_ = new value_type[capacity_];
00058         for(std::size_t i = 0; i < size_; i++)
00059         {
00060             data_[i] = other.data_[i];
00061         }
00062     }
00063     //move konstruktorius
00064     Vector(Vector&& other) noexcept
00065         :data_(other.data_),
00066         size_(other.size_),
00067         capacity_(other.capacity_)
00068     {
00069         other.data_ = nullptr;
00070         other.size_ = 0;
00071         other.capacity_ = 0;
00072     }
00073     //List konstruktorius
00074     Vector(std::initializer_list<value_type> init)
00075     {
00076         size_ = init.size();
00077         capacity_ = size_;
00078         data_ = new T[capacity_];
00079
00080         std::size_t i = 0;
00081         for(const T& value : init) {

```

```

00083         data_[i] = value;
00084         i++;
00085     }
00086 }
00087 //destruktorius
00088 ~Vector()
00089 {
00090     delete[] data_;
00091 }
00092
00093 //OPERATORS
00094 //copy assignment operator
00095 Vector& operator=(const Vector& other)
00096 {
00097     if(this == &other)
00098     {
00099         return *this;
00100     }
00101
00102     T* newBlock = new value_type[other.capacity_];
00103
00104     for(std::size_t i = 0; i < other.size_; i++)
00105     {
00106         newBlock[i] = other.data_[i];
00107     }
00108
00109     delete[] data_;
00110     data_ = newBlock;
00111     size_ = other.size_;
00112     capacity_ = other.capacity_;
00113
00114     return *this;
00115 }
00116 //move assignment operator
00117 Vector& operator=(Vector&& other) noexcept
00118 {
00119     if(this == &other) return *this;
00120     size_ = other.size_;
00121     capacity_ = other.capacity_;
00122     delete[] data_;
00123     data_ = other.data_;
00124
00125     other.data_ = nullptr;
00126     other.size_ = 0;
00127     other.capacity_ = 0;
00128
00129     return *this;
00130 }
00131
00132 //negalima keisti elemento
00133 const_reference operator[](size_type index) const
00134 {
00135     return data_[index];
00136 }
00137 //galima keisti elementa
00138 reference operator[](size_type index)
00139 {
00140     return data_[index];
00141 }
00142
00143 void resize(size_type newSize)
00144 {
00145     if(newSize > capacity_)
00146     {
00147         ReAlloc(newSize);
00148     }
00149     for(std::size_t i = size_; i < newSize; i++)
00150     {
00151         data_[i] = T{};
00152     }
00153
00154     size_ = newSize;
00155 }
00156
00157 void assign(size_type count, const_reference value)
00158 {
00159     delete[] data_;
00160     data_ = new value_type[count];
00161
00162     for(std::size_t i = 0; i < count; i++)
00163     {
00164         data_[i] = value;
00165     }
00166     size_ = count;
00167     capacity_ = count;
00168 }
00169

```

```

00170
00171 void push_back(const_reference value)
00172 {
00173     if(size_ >= capacity_)
00174     {
00175         //jei capacity 0, priskiriam 2, jei ne tada padvigubinam capacity
00176         ReAlloc(capacity_ == 0 ? 1 : capacity_ * 2);
00177     }
00178     //i tuscia vieta priskiriam elementa
00179     data_[size_] = value;
00180     size_++;
00181 }
00182
00183 void pop_back()
00184 {
00185     if(size_ > 0)
00186     {
00187         --size_;
00188     }
00189 }
00190
00191 void shrink_to_fit()
00192 {
00193     if(capacity_ > size_)
00194     {
00195         ReAlloc(size_);
00196     }
00197 }
00198
00199 void clear()
00200 {
00201     size_ = 0;
00202     capacity_ = 0;
00203     delete[] data_;
00204     data_ = nullptr;
00205 }
00206
00207 iterator insert(const_iterator pos, const_reference value)
00208 {
00209     std::size_t index = pos - begin();
00210
00211     if(size_ >= capacity_)
00212     {
00213         ReAlloc(capacity_ == 0 ? 2 : capacity_ * 2);
00214     }
00215     for(std::size_t i = size_; i > index; --i)
00216     {
00217         data_[i] = data_[i - 1];
00218     }
00219     data_[index] = value;
00220     size_++;
00221     return begin() + index;
00222 }
00223
00224 template<typename InputIt>
00225 iterator insert(const_iterator pos, InputIt first, InputIt last)
00226 {
00227     size_type index = pos - begin();
00228     size_type count = std::distance(first, last);
00229     while(size_ + count > capacity_)
00230     {
00231         ReAlloc(capacity_ == 0 ? 1 : capacity_ * 2);
00232     }
00233     for(size_type i = size_ + count - 1; i >= index + count; --i)
00234     {
00235         data_[i] = data_[i - count];
00236     }
00237     size_type i = index;
00238     for(auto it = first; it != last; ++it, ++i)
00239     {
00240         data_[i] = *it;
00241     }
00242     size_ += count;
00243     return begin() + index;
00244 }
00245
00246 iterator erase(const_iterator pos)
00247 {
00248     size_type index = pos - begin();
00249     for(std::size_t i = index; i < size_ - 1; i++)
00250     {

```

```

00257         data_[i] = data_[i + 1];
00258     }
00259     --size_;
00260
00261     return begin() + index;
00262 }
00263
00264 iterator erase(const_iterator first, const_iterator last)
00265 {
00266     size_type index_first = first - begin();
00267     size_type index_last = last - begin();
00268     size_type count = index_last - index_first;
00269
00270     for(size_type i = index_first; i < size_ - count; i++)
00271     {
00272         data_[i] = std::move(data_[i + count]);
00273     }
00274     size_ -= count;
00275     return begin() + index_first;
00276 }
00277
00278 void swap(Vector& other)
00279 {
00280     std::swap(data_, other.data_);
00281     std::swap(size_, other.size_);
00282     std::swap(capacity_, other.capacity_);
00283 }
00284
00285
00286
00287 //grazina rodykle i masyva
00288 pointer data() { return data_; }
00289 const_pointer data() const { return data_; }
00290
00291 //grazina elementa is duotos pozicijos
00292 reference at(size_type index)
00293 {
00294     if(index >= size_)
00295     {
00296         throw std::out_of_range("Indeksas už vektoriaus ribų!");
00297     }
00298     return data_[index];
00299 }
00300 //negalima keisti reiksmiu
00301 const_reference at(size_type index) const
00302 {
00303     if(index >= size_)
00304     {
00305         throw std::out_of_range("Indeksas už vektoriaus ribų!");
00306     }
00307     return data_[index];
00308 }
00309 //pirmas elementas
00310 reference front() { return data_[0]; }
00311 const_reference front() const { return data_[0]; }
00312
00313 //paskutinis elementas
00314 reference back() { return data_[size_ - 1]; }
00315 const_reference back() const { return data_[size_ - 1]; }
00316
00317 //ar tuscias
00318 bool empty() const { return size_ == 0; }
00319
00320 //grazina elemmentu kieki
00321 size_type size() const { return size_; }
00322
00323 //grazina talpa
00324 size_type capacity() const { return capacity_; }
00325
00326 //rezervuoja vieta
00327 void reserve(size_type new_cap)
00328 {
00329     if(new_cap > capacity_)
00330     {
00331         ReAlloc(new_cap);
00332     }
00333 }
00334
00335 //iteratoriai:
00336 iterator begin() { return data_; }
00337 const_iterator begin() const { return data_; }
00338
00339 iterator end() { return data_ + size_; }
00340 const_iterator end() const { return data_ + size_; }
00341
00342 //kai tikrai norime tik skaityti
00343 const_iterator cbegin() const noexcept { return data_; }

```



```

00344     const_iterator cend() const noexcept { return data_ + size_; }
00345 };
00346 };
00347
00348 //non-member functions
00349 template<typename T>
00350 bool operator==(const Vector<T>& lhs, const Vector<T>& rhs)
00351 {
00352     if(lhs.size() != rhs.size())
00353     {
00354         return false;
00355     }
00356     for(std::size_t i = 0; i < lhs.size(); i++)
00357     {
00358         if(lhs.at(i) != rhs.at(i))
00359         {
00360             return false;
00361         }
00362     }
00363     return true;
00364 }
00365
00366 template<typename T>
00367 bool operator!=(const Vector<T>& lhs, const Vector<T>& rhs)
00368 {
00369     if(lhs.size() != rhs.size())
00370     {
00371         return true;
00372     }
00373     for(std::size_t i = 0; i < lhs.size(); i++)
00374     {
00375         if(lhs.at(i) != rhs.at(i))
00376         {
00377             return true;
00378         }
00379     }
00380     return false;
00381 }
00382
00383 #endif

```

5.7 include/zmogus.h Failo Nuoroda

```

#include <string>
#include <iostream>

```

Įtraukimo priklausomybių diagrama zmogus.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Klasės

- class [Zmogus](#)

Abstrakti klasė sauganti vardą ir pavardę

5.8 zmogus.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00012 class Zmogus {
00013 protected:
00014     std::string vardas_;
00015     std::string pavarde_;
00016
00017 public:
00021     Zmogus() = default;
00027     Zmogus(const std::string& v, const std::string& p)

```

```

00028         : vardas_(v), pavarde_(p) {}
00029
00033     virtual ~Zmogus()
00034     {
00035         vardas_.clear();
00036         pavarde_.clear();
00037     };
00042     Zmogus(const Zmogus &a)
00043         : vardas_(a.vardas_),
00044           pavarde_(a.pavarde_) {}
00045
00051     Zmogus& operator=(const Zmogus &a)
00052     {
00053         if(this != &a)
00054         {
00055             vardas_ = a.vardas_;
00056             pavarde_ = a.pavarde_;
00057         }
00058         return *this;
00059     }
00060
00061
00066     Zmogus(Zmogus&& a)
00067         : vardas_(std::move(a.vardas_)),
00068           pavarde_(std::move(a.pavarde_)) {}
00069
00075     Zmogus& operator=(Zmogus&& a)
00076     {
00077         if(this != &a)
00078         {
00079             vardas_ = std::move(a.vardas_);
00080             pavarde_ = std::move(a.pavarde_);
00081         }
00082
00083         return *this;
00084     }
00085
00086
00091     virtual double vidurkis() const = 0;
00092
00093
00097     const std::string& getVardas() const { return vardas_; }
00101     const std::string& getPavarde() const { return pavarde_; }
00102
00107     void setVardas(const std::string& v) { vardas_ = v; }
00112     void setPavarde(const std::string& p) { pavarde_ = p; }
00113
00114 };
00115
00116 #endif

```

5.9 src/funkcijos.cpp Failo Nuoroda

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <fstream>
#include <chrono>
#include "studentas.h"
#include "funkcijos.h"
#include "vector.h"

```

Įtraukimo priklausomybių diagrama funkcijos.cpp:

Funkcijos

- void `spausdinimas (Vector< Studentas > &A)`
Spausdina studentų sąrašą
- void `failoSpausdinimas (Vector< Studentas > &A)`
Spausina studentų sąrašą ekrane arba faile.

- void `failoGeneravimas` (int studKiekis)
Generuoja studentų failą
- void `tyrimasPirmas` ()
Atlieka tyrimą - matuoja failų generavimo laiką
- void `tyrimasAntras` (Vector< Studentas > &A, Vector< Studentas > &vargsai, Vector< Studentas > &kietekai)
Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

5.9.1 Funkcijos Dokumentacija

5.9.1.1 failoGeneravimas()

```
void failoGeneravimas (
    int studKiekis )
```

Generuoja studentų failą

Parametrai

<code>studKiekis</code>	Studentų kiekis
-------------------------	-----------------

5.9.1.2 failoSpausdinimas()

```
void failoSpausdinimas (
    Vector< Studentas > & A )
```

Spausina studentų sąrašą ekrane arba faile.

Parametrai

<code>A</code>	Studentų vektorius
----------------	--------------------

5.9.1.3 spausdinimas()

```
void spausdinimas (
    Vector< Studentas > & A )
```

Spausdina studentų sąrašą

Parametrai

<code>A</code>	Studentų vektorius
----------------	--------------------

5.9.1.4 tyrimasAntras()

```
void tyrimasAntras (
```

```
Vector< Studentas > & A,
Vector< Studentas > & vargsai,
Vector< Studentas > & kietekai )
```

Atlieka tyrimą - matuoja nuskaitymo ir padalinimo laiką

Parametrai

<i>A</i>	Studentų vektorius
<i>vargsai</i>	Prastai besimokančių studentų vektorius
<i>kietekai</i>	Gerau besimokančių studentų vektorius

5.9.1.5 tyrimasPirmas()

```
void tyrimasPirmas ( )
```

Atlieka tyrimą - matuoja failų generavimo laiką

5.10 src/main.cpp Failo Nuoroda

```
#include <iostream>
#include <ctime>
#include <cstdlib>
#include <stdexcept>
#include <list>
#include <deque>
#include "funkcijos.h"
#include "studentas.h"
#include "zmogus.h"
#include "vector.h"
```

Itraukimo priklausomybių diagrama main.cpp:

Funkcijos

- int `main` ()

Kintamieji

- const `Vector`< std::string > `vardai` = {"Rokas", "Džiugas", "Kajus", "Dovydas", "Matas", "Simonas", "Mantas", "Kasparas", "Tomas", "Kristupas"}
- const `Vector`< std::string > `pavardes` = {"Brazdeikis", "Kazlauskas", "Macijauskas", "Sabonis", "Valanciunas", "Tubelis", "Sirvydis", "Ulanovas", "Giedraitis", "Kalnietis"}

5.10.1 Funkcijos Dokumentacija

5.10.1.1 main()

```
int main ( )
```

5.10.2 Kintamojo Dokumentacija

5.10.2.1 pavardes

```
const Vector<std::string> pavardes = {"Brazdeikis" , "Kazlauskas", "Macijauskas", "Sabonis",  
"Valanciunas", "Tubelis", "Sirvydis", "Ulanovas", "Giedraitis", "Kalnietis"}
```

5.10.2.2 vardai

```
const Vector<std::string> vardai = {"Rokas", "Dziugas", "Kajus", "Dovydas", "Matas", "Simonas",  
"Mantas", "Kasparas", "Tomas", "Kristupas"}
```


Rodyklė

~Vector
 Vector< T >, [10](#)
~Zmogus
 Zmogus, [16](#)

assign
 Vector< T >, [10](#)
at
 Vector< T >, [10](#)
atskiriFailai
 funkcijos.h, [20](#)

back
 Vector< T >, [11](#)
begin
 Vector< T >, [11](#)

capacity
 Vector< T >, [11](#)
cbegin
 Vector< T >, [11](#)
cend
 Vector< T >, [11](#)
clear
 Vector< T >, [11](#)
const_iterator
 Vector< T >, [8](#)
const_pointer
 Vector< T >, [8](#)
const_reference
 Vector< T >, [8](#)
const_reverse_iterator
 Vector< T >, [9](#)

data
 Vector< T >, [11](#), [12](#)
difference_type
 Vector< T >, [9](#)

empty
 Vector< T >, [12](#)
end
 Vector< T >, [12](#)
erase
 Vector< T >, [12](#)

failoGeneravimas
 funkcijos.cpp, [37](#)
 funkcijos.h, [20](#)
failoSpausdinimas
 funkcijos.cpp, [37](#)
 funkcijos.h, [21](#)
front
 Vector< T >, [12](#)
funkcijos.cpp
 failoGeneravimas, [37](#)
 failoSpausdinimas, [37](#)
 spausdinimas, [37](#)
 tyrimasAntras, [37](#)
 tyrimasPirmas, [38](#)
funkcijos.h
 atskiriFailai, [20](#)
 failoGeneravimas, [20](#)
 failoSpausdinimas, [21](#)
 KonteineriuTyrimas, [21](#)
 rikiavimas, [21](#)
 skaitymas, [22](#)
 spausdinimas, [22](#)
 StudentuPadalinimas, [22](#)
 StudentuPadalinimas2, [23](#)
 StudentuPadalinimas3, [23](#)
 tyrimasAntras, [23](#)
 tyrimasPirmas, [24](#)

getPavarde
 Zmogus, [17](#)
getVardas
 Zmogus, [17](#)

include/funkcijos.h, [19](#), [24](#)
include/studentas.h, [28](#)
include/vector.h, [30](#), [31](#)
include/zmogus.h, [35](#)
insert
 Vector< T >, [13](#)
iterator
 Vector< T >, [9](#)

KonteineriuTyrimas
 funkcijos.h, [21](#)

main
 main.cpp, [38](#)
main.cpp
 main, [38](#)
 pavardes, [39](#)
 vardai, [39](#)

operator!=
 vector.h, [30](#)

operator=
 Vector< T >, 13
 Zmogus, 17
 operator==
 vector.h, 30
 operator[]
 Vector< T >, 13

 pavarde_
 Zmogus, 18
 pavardes
 main.cpp, 39
 pointer
 Vector< T >, 9
 pop_back
 Vector< T >, 13
 push_back
 Vector< T >, 14

 reference
 Vector< T >, 9
 reserve
 Vector< T >, 14
 resize
 Vector< T >, 14
 reverse_iterator
 Vector< T >, 9
 rikiavimas
 funkcijos.h, 21

 setPavarde
 Zmogus, 18
 setVardas
 Zmogus, 18
 shrink_to_fit
 Vector< T >, 14
 size
 Vector< T >, 14
 size_type
 Vector< T >, 9
 skaitymas
 funkcijos.h, 22
 spausdinimas
 funkcijos.cpp, 37
 funkcijos.h, 22
 src/funkcijos.cpp, 36
 src/main.cpp, 38
 Studentas, 7
 StudentuPadalinimas
 funkcijos.h, 22
 StudentuPadalinimas2
 funkcijos.h, 23
 StudentuPadalinimas3
 funkcijos.h, 23
 swap
 Vector< T >, 14

 tyrimasAntras
 funkcijos.cpp, 37
 funkcijos.h, 23
 tyrimasPirmas
 funkcijos.cpp, 38
 funkcijos.h, 24

 value_type
 Vector< T >, 9
 vardai
 main.cpp, 39
 vardas_
 Zmogus, 18
 Vector
 Vector< T >, 10
 Vector< T >, 7
 ~Vector, 10
 assign, 10
 at, 10
 back, 11
 begin, 11
 capacity, 11
 cbegin, 11
 cend, 11
 clear, 11
 const_iterator, 8
 const_pointer, 8
 const_reference, 8
 const_reverse_iterator, 9
 data, 11, 12
 difference_type, 9
 empty, 12
 end, 12
 erase, 12
 front, 12
 insert, 13
 iterator, 9
 operator=, 13
 operator[], 13
 pointer, 9
 pop_back, 13
 push_back, 14
 reference, 9
 reserve, 14
 resize, 14
 reverse_iterator, 9
 shrink_to_fit, 14
 size, 14
 size_type, 9
 swap, 14
 value_type, 9
 Vector, 10
 vector.h
 operator!=, 30
 operator==, 30
 vidurkis
 Zmogus, 18

 Zmogus, 15
 ~Zmogus, 16
 getPavarde, 17

[getVardas, 17](#)
[operator=, 17](#)
[pavarde_, 18](#)
[setPavarde, 18](#)
[setVardas, 18](#)
[vardas_, 18](#)
[vidurkis, 18](#)
[Zmogus, 16](#)